

Proyecto 2 IA: Algoritmos de Búsqueda

Alina T. Carías (22539), Ignacio Méndez (22613), Ariela Mishaan (22052), Diego Soto (22737),
Ingeniería en Ciencia de los Datos, UVG

Abstract—This project presents a performance comparison of different search algorithms applied to solving randomly generated mazes. Maze generation is performed using minimum spanning tree algorithms. Subsequently, search algorithms are evaluated to find the shortest path between two points in a maze. Multiple simulations are conducted across different scenarios, measuring metrics such as execution time, number of explored nodes, and path length. The results provide insights into the efficiency and behavior of each algorithm under varying conditions, highlighting significant differences in performance.

Resumen— Este proyecto presenta una comparación del desempeño de distintos algoritmos de búsqueda aplicados a la resolución de laberintos generados aleatoriamente. Para la construcción de los laberintos se emplearon algoritmos de árboles de expansión mínima. Posteriormente, se evaluaron algoritmos de búsqueda de ruta más corta entre dos puntos del laberinto. Se realizaron múltiples simulaciones en distintos escenarios, midiendo métricas como tiempo de ejecución, número de nodos explorados y longitud del camino encontrado. Los resultados permiten analizar la eficiencia y comportamiento de cada algoritmo bajo diferentes configuraciones, evidenciando diferencias significativas en su desempeño.

Palabras clave—search algorithms, mazes, BFS, DFS, Dijkstra, A*, Kruskal, Prim.

I. INTRODUCCIÓN

La búsqueda de caminos óptimos en entornos discretos es un problema fundamental en la inteligencia artificial, con aplicaciones en robótica, videojuegos, navegación y sistemas de transporte. Los laberintos representan un entorno ideal para estudiar este tipo de problemas, ya que permiten modelar de forma clara la exploración de espacios y la toma de decisiones.

En este proyecto se aborda tanto la generación como la resolución de laberintos. Primero, se construyen laberintos aleatorios mediante algoritmos de árboles de expansión mínima, los cuales garantizan conectividad sin ciclos redundantes. Luego, se aplican distintos algoritmos de búsqueda para encontrar rutas entre un punto de inicio y uno de destino.

El objetivo principal es comparar el desempeño de varios algoritmos de búsqueda – BFS, DFS, Dijkstra y A* – en distintos escenarios, evaluando métricas como el número de nodos explorados, el tiempo de ejecución y longitud de la ruta encontrada. A través de esta comparación, se busca identificar las ventajas y limitaciones de cada enfoque, así como su

comportamiento frente a diferentes configuraciones de laberintos.

II. MARCO TEÓRICO

A. Generación de laberintos

Algoritmo de Kruskal

El algoritmo de Kruskal es un método para construir un árbol de expansión mínima, es decir, una estructura que conecta todos los nodos sin formar ciclos y con el menor número de conexiones necesarias. En el caso de los laberintos, esto se traduce en crear caminos sin bucles redundantes, garantizando que exista una única ruta entre dos puntos.

Para generar el laberinto, se parte de una grilla donde cada celda está completamente aislada por paredes. Luego, se consideran todas las posibles paredes entre celdas vecinas como conexiones candidatas. El algoritmo selecciona estas paredes en orden aleatorio y decide eliminarlas únicamente si al hacerlo no se forma un ciclo, lo cual se controla agrupando las celdas en conjuntos disjuntos.

De manera intuitiva, el proceso consiste en ir “rompiendo paredes” poco a poco, conectando celdas que antes estaban separadas, hasta que todo el laberinto queda conectado. El resultado es un laberinto con una estructura bastante uniforme, donde no existen caminos redundantes y todas las regiones están conectadas (Senturk, 2024).

Algoritmo de Prim

El algoritmo de Prim también genera un árbol de expansión mínima, pero en lugar de comenzar con todas las conexiones posibles, empieza desde una sola celda y va expandiendo el laberinto progresivamente.

El proceso inicia seleccionando una celda inicial y marcándola como parte del laberinto. Luego, se agregan a una lista las paredes que conectan esta celda con sus vecinos. En cada iteración, se elige una de estas paredes de forma aleatoria; si la celda al otro lado aún no ha sido visitada, se elimina la pared y se incorpora esa nueva celda al laberinto. A su vez, se añaden las nuevas paredes de esta celda a la lista de candidatos.

Este procedimiento se repite hasta que todas las celdas han sido visitadas. A diferencia de Kruskal, aquí el laberinto crece como una especie de mancha que se expande desde el punto inicial. Como resultado, los laberintos generados con Prim tienden a tener caminos más orgánicos y ramificados, con mayor concentración de estructuras alrededor del punto de inicio (Zong, 2012).

B. Algoritmos de búsqueda de rutas

Breadth-First Search (BFS)

El algoritmo BFS (búsqueda en anchura) explora el laberinto por niveles, revisando primero todas las celdas cercanas al punto inicial antes de avanzar hacia zonas más lejanas. Utiliza una cola (FIFO) para almacenar las celdas por explorar, agregando en cada paso los vecinos no visitados. De esta manera, el recorrido se expande de forma uniforme en todas las direcciones desde el punto de inicio.

Su principal ventaja es que garantiza encontrar el camino más corto en términos de número de pasos cuando todos los movimientos tienen el mismo costo, como en un laberinto sin pesos. Sin embargo, puede resultar costoso en términos de exploración, ya que tiende a visitar un gran número de celdas antes de llegar a la solución (GeeksForGeeks, 2026).

Depth-First Search (DFS)

El algoritmo DFS (búsqueda en profundidad) se basa en explorar un camino lo más lejos posible antes de retroceder. A diferencia de BFS, no avanza por niveles, sino que sigue una ruta hasta encontrar un callejón sin salida, momento en el que regresa (backtracking) para intentar otras alternativas. Este comportamiento se implementa comúnmente mediante pila o recursión.

Aunque DFS es sencillo de implementar y requiere menos memoria que BFS, no garantiza encontrar el camino más corto. Dependiendo del orden de exploración, puede recorrer largas rutas innecesarias antes de encontrar la salida, lo que afecta su eficiencia en la resolución de laberintos (GeeksforGeeks, 2026).

Dijkstra (Cost Uniform Search)

El algoritmo de Dijkstra encuentra la ruta de menor costo considerando la distancia acumulada desde el punto inicial hasta cada celda. Para ello, utiliza una cola de prioridad que siempre selecciona la celda con el menor costo total conocido, permitiendo explorar primero las rutas más prometedoras en términos de distancia recorrida.

Este algoritmo garantiza encontrar el camino óptimo incluso cuando los movimientos tienen diferentes costos. En el caso de laberintos sin pesos, su comportamiento es similar al de BFS, aunque con un mayor costo computacional debido a la gestión de prioridades en la exploración (Fogler, 2023).

*A**

El algoritmo A* es una mejora de Dijkstra que incorpora una heurística para estimar la distancia restante hasta el objetivo. En cada paso, evalúa las celdas combinando el costo real desde el inicio con una estimación del costo hacia la meta, lo que permite priorizar rutas más prometedoras.

Gracias a esta estrategia, A* suele explorar menos nodos que Dijkstra y mantiene la capacidad de encontrar la solución óptima siempre que la heurística sea adecuada. Por esta razón, es uno de los algoritmos más eficientes y utilizados para resolver problemas de búsqueda de caminos en laberintos (Tosato, 2024).

III. DISEÑO EXPERIMENTAL

A. Representación del entorno

La implementación del proyecto se basa en una clase *Maze*, que representa el laberinto como una matriz donde las celdas corresponden a espacios transitables y las paredes a obstáculos. Esta estructura permite modelar el entorno de forma sencilla y eficiente para la ejecución de los algoritmos de generación y búsqueda.

Además, la clase incluye métodos auxiliares para convertir coordenadas de celdas a posiciones dentro de la matriz, eliminar paredes entre celdas y registrar el historial de construcción del laberinto. Este último aspecto es clave para generar visualizaciones del proceso de generación.

B. Generación de laberintos

Para la construcción de los laberintos se implementaron los algoritmos de Kruskal y Prim, ambos basados en árboles de expansión mínima. En el caso de Kruskal, se utilizó una estructura *UnionFind* con compresión de caminos y unión por rango, lo que permite gestionar conjuntos disjuntos y evitar la formación de ciclos al eliminar paredes. Por otro lado, el algoritmo de Prim se implementó mediante una expansión incremental, utilizando una lista de paredes candidatas que se seleccionan aleatoriamente para ir incorporando nuevas celdas al laberinto. Ambos enfoques garantizan que el laberinto final sea completamente conexo y sin ciclos redundantes.

C. Implementación de algoritmos de búsqueda

Para resolver los laberintos se implementaron los algoritmos BFS, DFS, Dijkstra y A*. Todos ellos utilizan una función auxiliar *get_neighbors*, encargada de determinar las celdas accesibles desde una posición dada sin atravesar paredes.

DFS y BFS se implementaron utilizando estructuras de datos clásicas: cola para BFS y pila para DFS. En cambio, Dijkstra y A* emplean colas de prioridad, permitiendo seleccionar el siguiente nodo a explorar en función del costo acumulado o de una función heurística en el caso de A*.

D. Obtención de métricas

Cada algoritmo devuelve tanto la ruta encontrada como el conjunto de nodos explorados durante la ejecución. Esto permite analizar no solo la solución final, sino también el comportamiento interno de cada método.

A partir de estos resultados, se calcularon métricas clave como la longitud del camino encontrado, el número de nodos explorados y el tiempo de ejecución, las cuales son utilizadas posteriormente para la comparación entre algoritmos.

E. Visualización de resultados

Se desarrollaron funciones específicas como *plot_solution* y *animate_solution* para representar gráficamente los resultados. Estas visualizaciones muestran el laberinto, las celdas exploradas por cada algoritmo y el camino final encontrado.

Las animaciones permiten observar de forma dinámica cómo cada algoritmo explora el espacio, facilitando una comparación cualitativa de sus estrategias de búsqueda.

F. Configuración experimental

El diseño experimental se dividió en tres escenarios principales. En primer lugar, se resolvió un laberinto de tamaño 60x80 con posiciones fijas de inicio y fin, lo que permite comparar directamente el desempeño de los algoritmos en un mismo entorno. En segundo lugar, se realizaron animaciones del proceso de exploración para analizar visualmente el comportamiento de los algoritmos durante la búsqueda.

G. Evaluación en múltiples escenarios

Finalmente, se generaron $K = 25$ laberintos aleatorios de tamaño 45x55, utilizando los algoritmos de Kruskal y Prim. Para cada laberinto, se seleccionaron posiciones iniciales y finales de forma aleatoria, asegurando una distancia Manhattan mínima de 10 unidades entre ellas.

En cada uno de estos escenarios se ejecutaron los cuatro algoritmos de búsqueda, registrando métricas como longitud del camino, nodos explorados y tiempo de ejecución. Con base a estos resultados, se construyeron tablas comparativas y rankings de desempeño, permitiendo evaluar de manera cuantitativa la eficiencia de cada algoritmo en distintos contextos.

IV. RESULTADOS

Para la generación de laberinto se implementaron 2 algoritmos distintos, el algoritmo Kruskal y Prim sobre grids de tamaño 20x25. Para ambos laberintos se logró construir laberintos completamente conexos y sin ciclos repetitivos, garantizando la existencia de una única ruta para cualquier par de celdas.

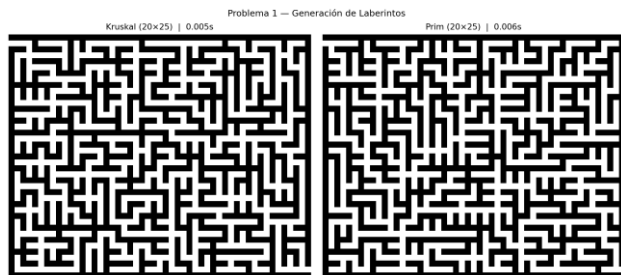


Figura 1- Generación de laberintos

En la Figura 1 se observa la comparación visual entre ambos algoritmos en la generación del laberinto. El algoritmo Kruskal generó estructuras con caminos más distribuidos y homogéneos a lo largo del grid, mientras que Prim generó patrones más orgánicos y con puntos de expansión más marcados. En ambos casos, los tiempos de generación fueron bajos, en el caso de Kruskal se tardó 0.005s y para Prim 0.006s, esto demuestra que ambos métodos son eficientes en la generación de laberintos.

Además de la conectividad total, los laberintos generados presentan diferencias visual en la distribución de pasillos y

ramificaciones. Kruskal genera caminos más balanceados y uniformes, mientras que Prim genera trayectorias más largas y agrupadas.

Para evaluar el desempeño de los algoritmos de búsqueda, se resolvieron laberintos de tamaño 60x80 usando BFS, DFS, Dijkstra y A*. En todos los casos, el punto inicial fue el punto (1,1) y el punto final la celda (60,80).

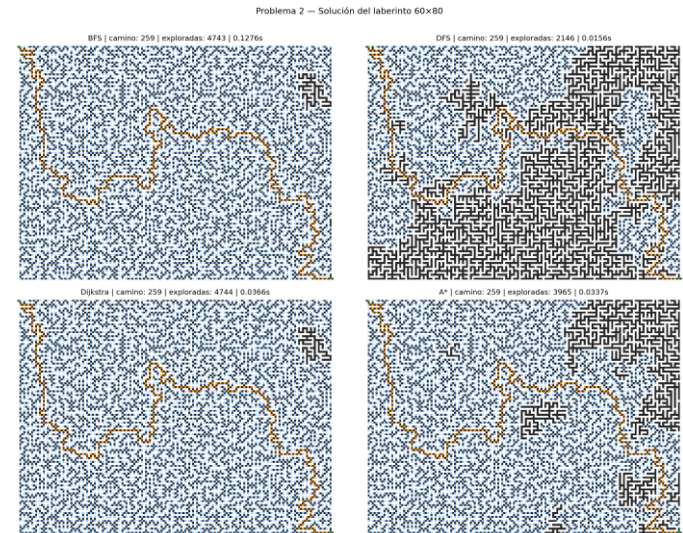


Figura 2 - Comparación algoritmos de búsqueda

La Figura 2 muestra las regiones exploradas y el camino encontrado para cada algoritmo. Todos los algoritmos lograron encontrar una ruta válida, en este caso ambos algoritmos encontraron la ruta con longitud 259. Sin embargo, el comportamiento de exploración presentó diferencias significativas.

BFS exploró 4743 nodos y registró el mayor tiempo de ejecución, con aproximadamente 0.1276s. De forma similar, Dijkstra exploró 4744 nodos, obteniendo prácticamente un comportamiento igual a BFS debido a que el laberinto no posee pesos distintos.

En el caso de DFS el comportamiento si es distinto ya que solamente exploró 2146 nodos y redujo el tiempo de ejecución a 0.0156s. Sin embargo, este algoritmo no garantiza eficiencia consistentes en todos los escenarios. Por su parte, A* exploró 3965 nodos y obtuvo un tiempo de ejecución de 0.0337.

Visualmente puede observarse que BFS y Dijkstra expanden regiones muy amplias del laberinto, mientras que DFS y A* concentra la exploración alrededor de trayectorias más prometedoras.

Con el objetivo de comparar el desempeño de los algoritmos en múltiple escenarios, se generaron $K = 25$ laberintos aleatorios de tamaño 45x55. Para cada escenario se seleccionamos posiciones iniciales y finales aleatorias, garantizando una distancia Manhattan mínima de 10 unidades.

En cada simulación se registraron tres métricas principales: longitud del camino encontrado, cantidad de nodos explorados y tiempo de ejecución.

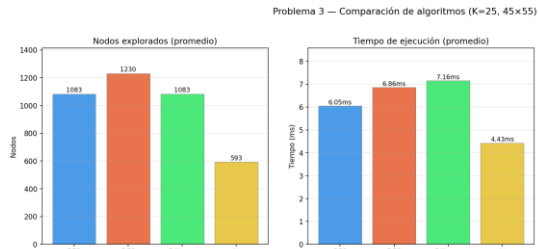


Figura 3 - Comparación algoritmos

La Figura 3 presenta el promedio de nodos explorados y tiempos de ejecución obtenidos por los cuatro algoritmos.

A* obtuvo el mejor desempeño global, explorando en promedio 592.6 nodos y registran el menor tiempo promedio de ejecución con 4.432ms. Además, alcanzó el primer lugar en 17 de los 25 escenarios evaluados.

BFS y Dijkstra presentaron comportamientos muy similares. BFS exploró en promedio 1083.3 nodos y Dijkstra 1082.9 nodos. Esto es esperado ya que el problema no incorpora pesos distintos entre movimientos. Sin embargo, Dijkstra registró el mayor tiempo promedio de ejecución con 7.158 ms debido al uso de colas de prioridad.

DFS exploró la mayor cantidad promedio de nodos con 1230.4 y obtuvo un tiempo promedio de 6.861 ms. Aunque en algunos escenarios logró muy buenos resultados, su desempeño fue altamente variable dependiendo de la estructura del laberinto y el orden de exploración.

Algoritmo	Rank promedio	Camino promedio	Nodos explorados promedio	Tiempo promedio
A*	1.32	81	592.6	4.432ms
DFS	2.56	81	1230.4	6.861ms
BFS	3.0	81	1083.3	6.054ms
Dijkstra	3.12	81	1082.9	7.158ms

Tabla 1 – Ranking promedio por algoritmo

Los resultados de la Tabla 1 muestran que todos los algoritmos encontraron caminos de longitud promedio 81, indicando que las rutas obtenidas fueron óptimas en la mayoría de los casos. Sin embargo, las diferencias principales surgen en la eficiencia de exploración y el tiempo para encontrar la ruta.

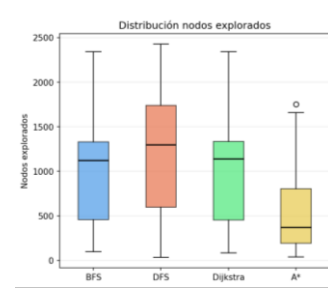


Figura 4 - Distribución de nodos

La Figura 4 presenta diagramas de caja con la distribución de nodos explorados por cada algoritmo a lo largo de los 25 escenarios.

A* mostró no solo el menor promedio de exploración, sino también una distribución más concentrada, evidenciando un comportamiento más estable y consistente. BFS y Dijkstra presentaron distribuciones prácticamente iguales, reforzando la equivalencia de ambos algoritmos en entornos sin pesos.

DFS mostró la mayor variabilidad entre escenarios. En algunos casos exploró muy pocos nodos y obtuvo excelentes resultados, mientras que en otros expandió regiones considerablemente mayores del laberinto. Esto confirma que su desempeño depende de la estructura del laberinto.

Con el fin de complementar el análisis cuantitativo, se seleccionaron distintos escenarios representativos para comparar visualmente la exploración realizada por cada algoritmo.

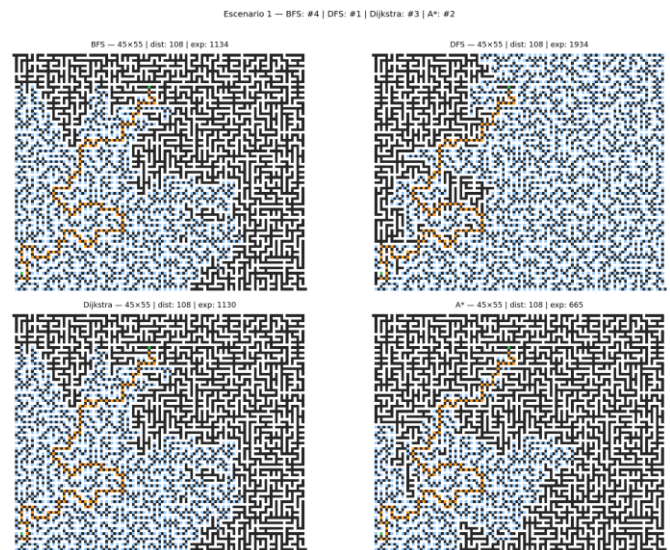


Figura 5 - Escenario 1

En la Figura 1 la cual representa el escenario 1, A* exploró 665 nodos, mientras que DFS exploró 1934. BFS y Dijkstra mantuvieron un comportamiento muy similar con aproximadamente 1130 nodos explorados. En este caso A* logró concentrar la búsqueda alrededor de la ruta óptima utilizando la heurística de Manhattan.

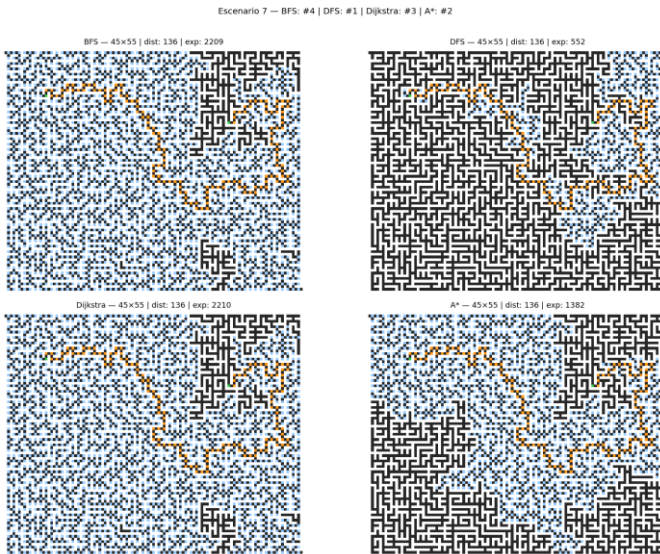


Figura 6 - Escenario 7

En la Figura 6 se presenta el escenario 7 donde ocurrió un comportamiento diferente. En este caso DFS exploró únicamente 552 nodos, superando incluso a A*. Esto evidencia que DFS puede obtener resultados sobresalientes cuando el orden de exploración coincide con la dirección de la solución.

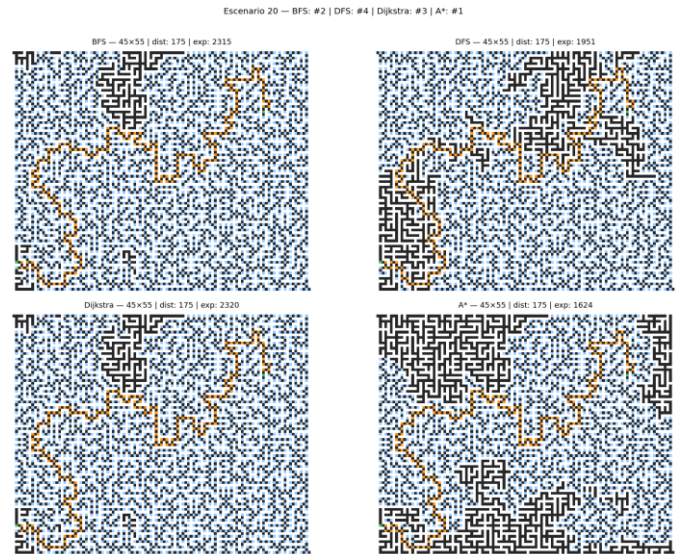


Figura 8 - Escenario 20

En la Figura 8 se presenta el escenario 20, en este todos los algoritmos necesitaron explorar regiones considerablemente mayores del laberinto debido a la complejidad estructural del escenario. BFS y Dijkstra superaron los 2300 nodos explorados, DFS alcanzó 1951 y A* redujo la exploración a 1624 nodos, manteniendo nuevamente el mejor desempeño.

Estos escenarios demuestran que A* mantiene un comportamiento consistente en la mayoría de las configuraciones de laberintos, mientras que DFS puede presentar resultados muy variables dependiendo de la forma del laberinto.

V. DISCUSIÓN DE RESULTADOS

Los resultados obtenidos muestran que los algoritmos de generación de laberintos, Kruskal y Prim, son eficientes y adecuados para este tipo de problemas. Ambos lograron construir estructuras completamente conexas y sin ciclos, cumpliendo con las propiedades de los árboles de expansión mínima. Sin embargo, se observaron diferencias en la forma de los laberintos, ya que Kruskal genera distribuciones más uniformes mientras que Prim produce patrones más orgánicos y concentrados.

Estas diferencias estructurales influyen directamente en el desempeño de los algoritmos de búsqueda. En particular, la distribución de caminos y ramificaciones puede facilitar o dificultar la exploración dependiendo del algoritmo utilizado. Esto evidencia que no solo el algoritmo de búsqueda importa, sino también la estructura del entorno en el que se aplica.

En cuanto a los algoritmos de búsqueda, BFS y Dijkstra presentaron comportamientos prácticamente idénticos en términos de nodos explorados. Esto es consistente con la teoría, ya que en ausencia de pesos ambos algoritmos funcionan de manera equivalente. No obstante, Dijkstra registró mayores tiempos de ejecución debido al uso de colas de prioridad, lo que introduce un costo adicional innecesario en este contexto.

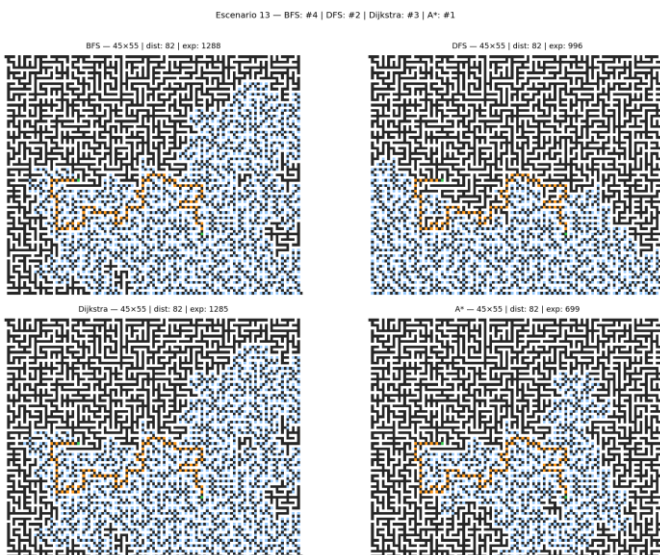


Figura 7 - Escenario 13

En la Figura 7 se presenta el escenario 13, donde A* volvió a posicionarse como el mejor algoritmo al explorar 699 nodos, mientras que BFS y Dijkstra superaron los 1280 nodos explorados. DFS obtuvo un desempeño intermedio con 996 nodos.

Por su parte, DFS mostró un comportamiento variable a lo largo de los escenarios evaluados. En algunos casos logró explorar menos nodos y obtener tiempos bajos, pero en otros su desempeño fue considerablemente peor. Esto confirma que su eficiencia depende fuertemente del orden de exploración y de la estructura del laberinto, lo que limita su confiabilidad.

El algoritmo A* destacó como el más eficiente y consistente en la mayoría de los escenarios. Gracias al uso de la heurística de Manhattan, logró reducir significativamente el número de nodos explorados en comparación con los demás algoritmos. Además, presentó una menor variabilidad en su desempeño, lo que lo convierte en una opción robusta.

En el análisis de los 25 escenarios, todos los algoritmos encontraron caminos de igual longitud, lo que indica que las soluciones fueron óptimas en la mayoría de los casos. Sin embargo, las diferencias en eficiencia fueron claras, siendo A* el mejor clasificado en la mayoría de las pruebas. BFS y Dijkstra confirmaron su equivalencia, mientras que DFS presentó la mayor variabilidad en resultados.

VI. CONCLUSIONES

Se concluye que los algoritmos de generación de laberintos, Kruskal y Prim, son adecuados para este tipo de aplicaciones, ya que garantizan conectividad y ausencia de ciclos. Sin embargo, la estructura que generan es diferente y puede afectar el desempeño de los algoritmos de búsqueda. Por lo tanto, la elección del método de generación también es relevante en este tipo de análisis.

En relación con los algoritmos de búsqueda, se comprobó que BFS y Dijkstra son equivalentes en entornos sin pesos. Sin embargo, BFS resulta más eficiente debido a su menor costo computacional. Esto hace que Dijkstra no sea la mejor opción en este tipo de problemas específicos.

DFS no garantiza un desempeño consistente, ya que su eficiencia depende de factores como la estructura del laberinto y el orden de exploración. Aunque puede ser eficiente en algunos casos, no es confiable como método general para encontrar rutas óptimas. Por esto, su uso debe considerarse con precaución.

Finalmente, A* se identificó como el algoritmo más eficiente y estable en todos los escenarios evaluados. Su capacidad para reducir la exploración mediante el uso de heurísticas lo convierte en la mejor alternativa para problemas de búsqueda de rutas.

BIBLIOGRAFÍA

- [1] B. Senturk (17 de julio, 2024), "Kruskal's Algorithm: The Most Visually Satisfying Maze Generation," in *Medium*, <https://medium.com/@batu.senturk/kruskals-algorithm-the-most-visually-satisfying-maze-generation-c5c565844a4e>
- [2] Maze Generation with Prim's Algorithm. (2012, 6 noviembre). Jonathan Zong. <https://jonathanzong.com/blog/2012/11/06/maze-generation-with-prims-algorithm>
- [3] Mitic, N. (2025, 31 julio). Comparing BFS, DFS, Dijkstra, and A* algorithms on a practical maze solver example - Nemanja Mitic. <https://nemanjamitic.com/blog/2025-07-31-maze-solver/#bfs>
- [4] Tosato, M. (2024, 9 junio). Exploring the Depths: Solving Mazes with A* Search Algorithm. <https://matteo-tosato7.medium.com/exploring-the-depths-solving-mazes-with-a-search-algorithm-c15253104899>
- [5] Fogler, O. (2023, 30 diciembre). Navigating the Maze: A Dive into Dijkstra's Algorithm for Finding Shortest Paths. <https://medium.com/@foglerori/navigating-the-maze-a-dive-into-dijkstras-algorithm-for-finding-shortest-paths-7ef3aaa89bd5>
- [6] GeeksforGeeks. (2026b, marzo 28). Depth First Search or DFS for a Graph. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
- [7] GeeksforGeeks. (2026b, enero 16). Breadth First Search or BFS for a Graph. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>